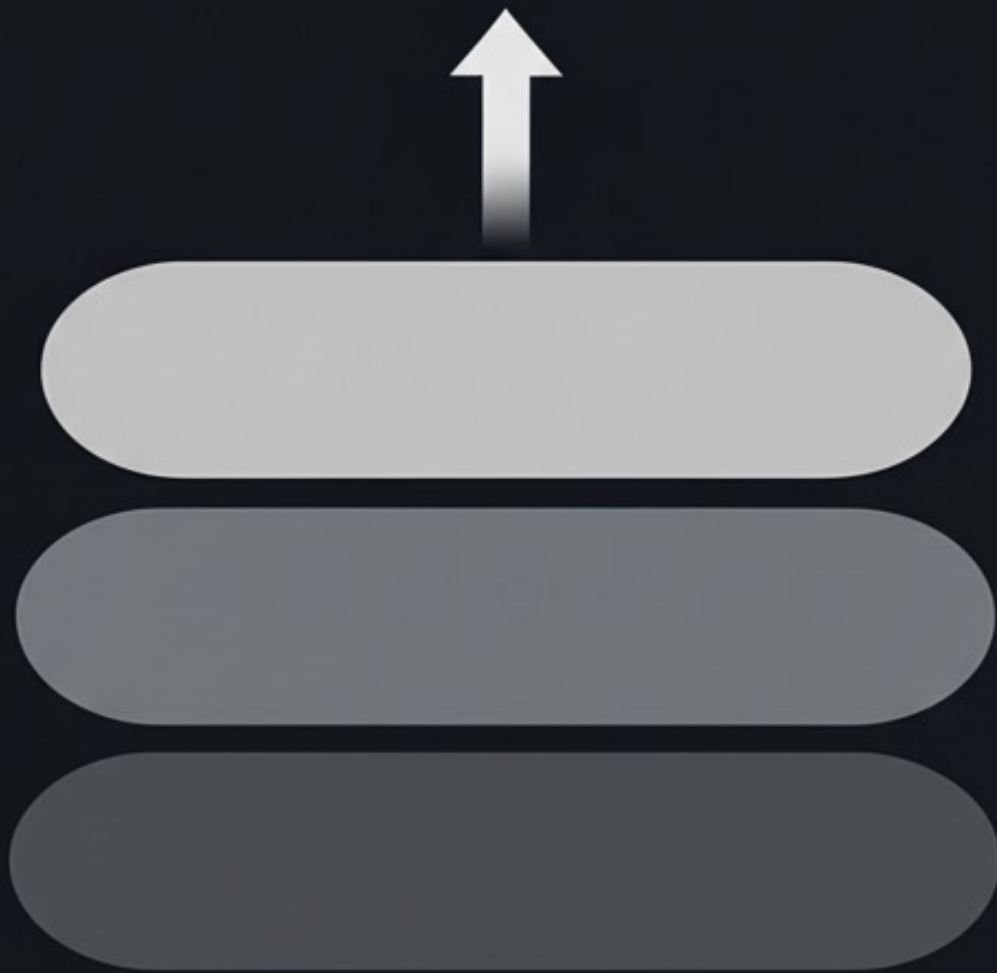


DATA STRUCTURES AND ALGORITHMS



STACK

C {...}

Introduction

LIFO : Last In First Out

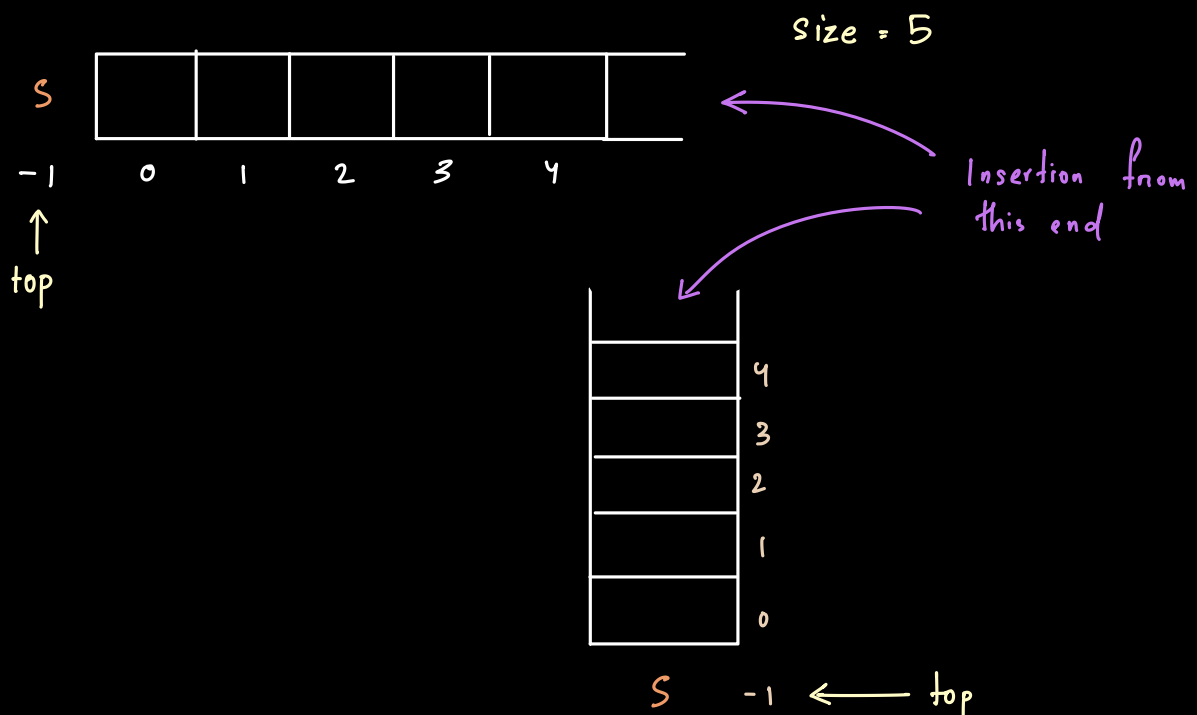
ADT STACK

	4
5	3
2	2
16	1
10	0

Data : 1. Space for storing elements
2. Top pointer

Operations : 1. push(x)
2. pop()
3. peek(Index)
4. stackTop()
5. isEmpty()
6. isFull()

(1) Stack using Array



```

struct Stack {
    int size ;
    int Top ;
    int *s ;      → pointer to an array
} ;

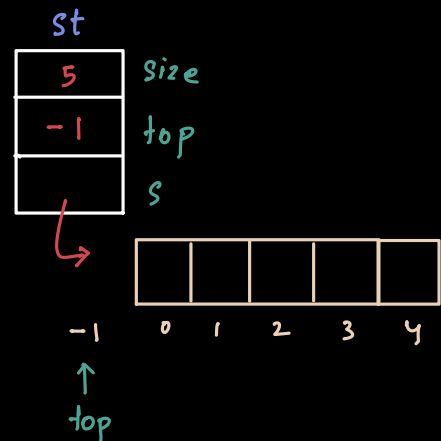
```

```

int main() {
    struct Stack st ;
    printf("Enter size of stack :");
    scanf("%d", &st.size);

    st.s = new int [st.size];
    st.Top = -1 ;
}

```



◦ Empty

if (`Top == -1`)

◦ Full

if (`Top == size - 1`)

◦ Push

→ time complexity : constant

```

void push (stack *st, int x) {
    if (st->Top == st->size - 1)
        printf("Stack Overflow");
    else {
        st->Top ++ ;
        st->s[st->Top] = x ;
    }
}

```

o pop

→ time complexity : constant

```
int push ( stack *st ) {  
    int x = -1 ;  
    if ( st->Top == -1 )  
        printf ( "Stack underflow" ) ;  
    else {  
        x = st->s [ st->Top ] ;  
        st->Top -- ;  
    }  
    return x ;  
}
```

o peek

→ time complexity : constant

	4
5	3 ← top
2	2
16	1
10	0
S	-1

pos	Index = top - pos + 1
1	3 = 3 - 1 + 1
2	2 = 3 - 2 + 1
3	1 = 3 - 3 + 1
4	0 = 3 - 4 + 1

```
int peek ( stack *st, int pos ) {  
    int x = -1 ;  
    if ( st->Top - pos + 1 < 0 )  
        printf ( "pos invalid " ) ;  
}
```



```

    else {
        x = st.s[st.Top - pos + 1];
    }
    return x;
}

```

0 stackTop

→ const. time

```

int stackTop(stack st) {
    if (st.Top == -1)
        return -1;
    else
        return st.s[st.Top];
}

```

0 is Empty

→ const. time

```

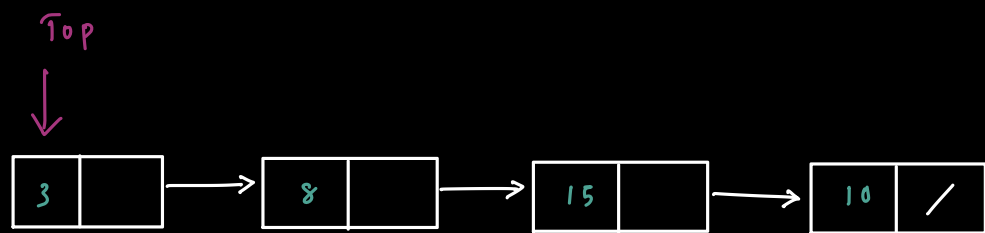
int isEmpty(stack st) {
    if (st.Top == -1)
        return 1;
    else
        return 0;
}

```

0 is Full $\rightarrow$ const. time

```
int isFull (stack st) {  
    if (st.Top == st.size - 1)  
        return 1;  
    else  
        return 0;  
}
```

(2) Stack using Linked List



```
struct Node {  
    int data;  
    struct Node * next;  
}
```

0 Insertion & Deletion $\rightarrow$ $O(1)$

◦ Empty

```
if (Top == NULL)
```

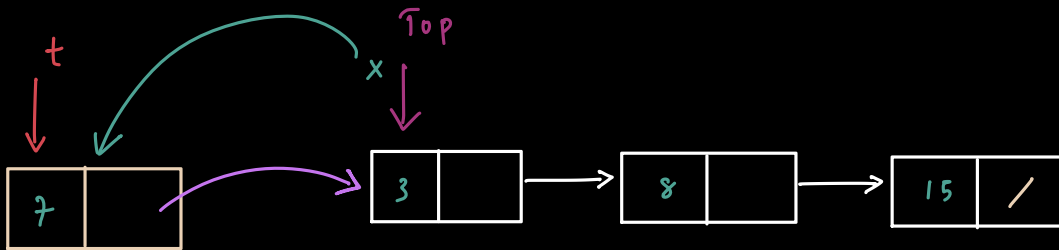
◦ Full

```
Node *t = new Node;
```

```
if (t == NULL)
```

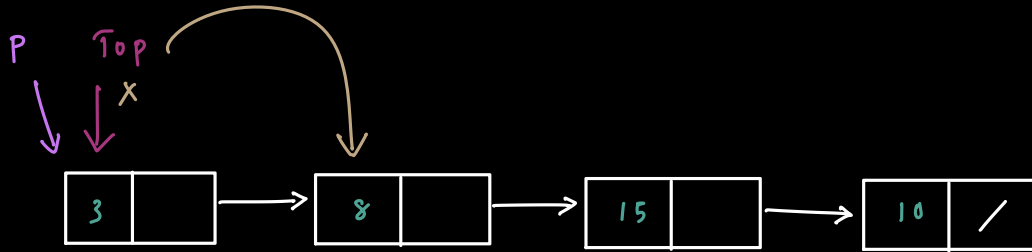
Stack never gets full until all spaces in heap is utilised.

◦ Push



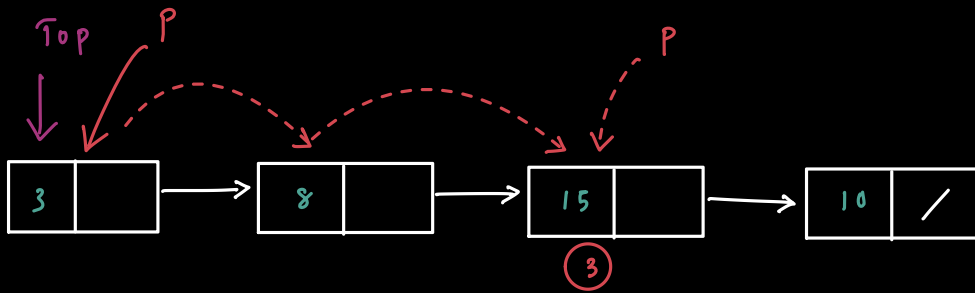
```
void push ( int n ) {  
    Node *t = (Node *) malloc ( sizeof ( Node ) );  
    if ( t == NULL )  
        printf ( " Stack Overflow " );  
    else {  
        t -> data = n ;  
        t -> next = Top ;  
        Top = t ;  
    }  
}
```

o pop



```
int pop ( ) {  
    Node *p ;  
    int n = -1 ;  
    if ( Top == NULL )  
        printf ( "Stack is empty" ) ;  
    else {  
        p = Top ;  
        Top = Top -> next ;  
        n = p -> data ;  
        free ( p ) ;  
    }  
    return n ;  
}
```

o peak



```
int peek ( int pos ) {
```

```
    Node *p = Top ;
```

```
    for ( int i = 0 ; p != NULL && i < pos - 1 ; i++ ) {
```

```
        p = p → next ;
```

```
    }
```

```
    if ( p != NULL )
```

```
        return p → data ;
```

```
    else
```

```
        return -1 ;
```

```
}
```

o starkTop

```
int starkTop () {
```

```
    if ( Top )
```

```
        return Top → data ;
```

```
    return -1 ;
```

```
}
```

◦ is Empty

```
int isEmpty () {  
    return Top ? 0 : 1 ;    // If top is NULL → empty  
}
```

◦ is Full

```
int isFull () {  
    Node *t = (Node *) malloc (sizeof (Node)) ;  
    int r = t ? 0 : 1 ;  
    free (t) ;  
    return r ;  
}
```

◦ Parenthesis matching

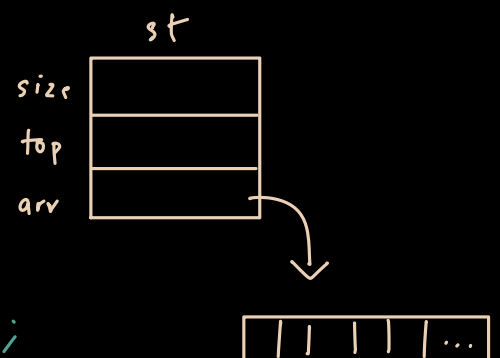
```
struct stack {  
    int size ;  
    int Top ;  
    char *arr ;  
}
```

exp

<	<	a	+	b	)	*	<	)	\0
0	1	2	3	4	5	6	7	8	9

// As string is being passed

```
int isBalance (char *exp) {  
    struct stack st ;  
    st.size = strlen (exp) ;  
    st.top = -1 ;  
    st.arr = new char [st.size] ;
```



Expression Conversion

① Infix : Operand Operator Operand

eg : $a + b$

② Prefix : Operator Operand Operand

eg. $+ a b$

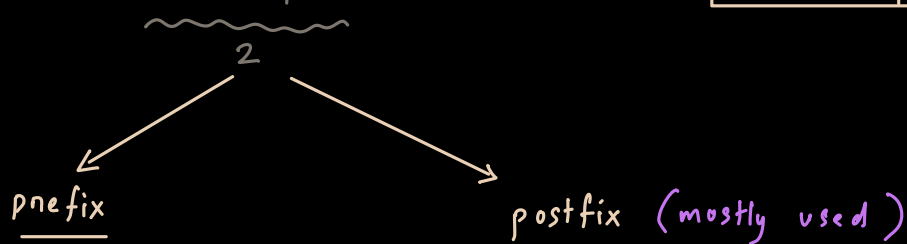
③ Postfix : Operand Operand Operator

eg. $a b +$

1. Infix to Postfix Expression

① $a + b * c$

$$a + (b * c)$$
$$\equiv (a + (b * c))$$



symbol	Precedence	Associ.
$+, -$	1	L-R
$*, /$	2	L-R
$(,)$	3	

$$\equiv (a + (* b c))$$

$$\equiv + a * b c$$

$$\equiv (a + (b c *))$$

$$\equiv a b c * +$$

② $a + b + c * d$

prefix

postfix

$\equiv a + b + (*cd)$

$\equiv a + b + (cd*)$

when operators of same precedence are there then take left to right.

$\equiv (ab+) + (cd*)$

$\equiv (+ab) + (*cd)$

$\equiv ab + cd * +$

$\equiv ++ab * cd$

METHOD 1

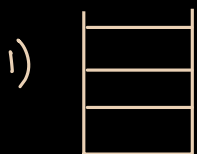
ex: $a + b * c - d / e \rightarrow$ Infix

symbol	Precedence	Associ.
$+, -$	1	L-R
$*, /$	2	L-R

stack

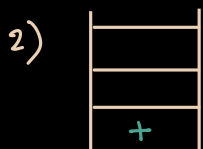
Push/pop

postfix



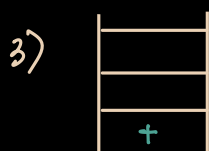
—

a



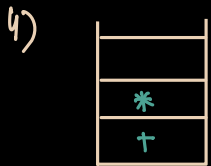
push +

a



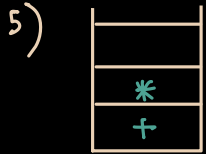
—

ab



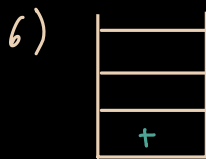
push *
(as * has ↑
precedence)

ab



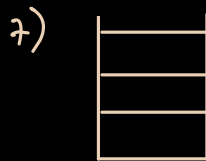
—

abc



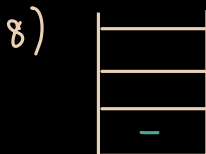
pop *
(as + has ↓
precedence
than *)

abc *



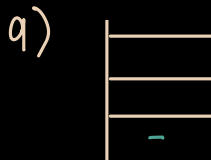
pop +

abc * +



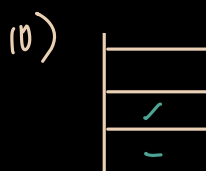
push -

abc * +



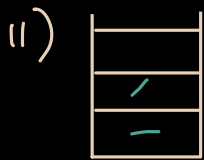
—

abc * + d

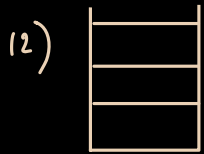


push /

abc * + d



$abc * + de$



pop / and
then -

$abc * + de / -$

METHOD 2

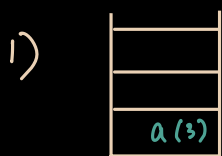
exp: $a + b * c - d / e$

Symbol	Precedence	Associ.
$+, -$	1	L-R
$*, /$	2	L-R
$a, b, c, \dots$	3	L-R

Stack

Push/pop

Postfix

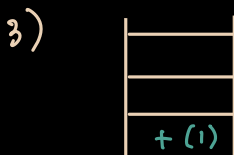


push a



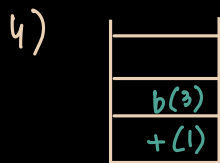
pop a

a



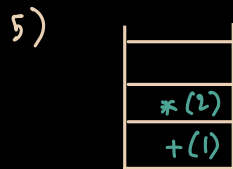
push +

a



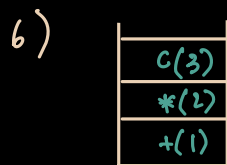
push b

a



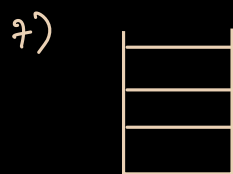
pop b

ab



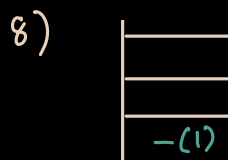
push c

abc



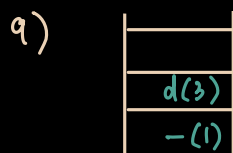
pop c then
* and then +

abc*+



push -

abc*+



push d

abc*+



pop d

abc*+d

11)

/ (2)
- (1)

 push / abc * + d

12)

e (3)
/ (2)
- (1)

 push e abc * + d

13)

 pop e, / and - abc * + de / -

Program

```

struct stack {
    int size ;
    int Top ;
    char *arr ;
}

int isOperand ( char x ) {
    if ( x == '+' || x == '-' || x == '*' || x == '/' )
        return 0 ;
    else
        return 1 ;
}
    
```

```

int precedence (char x) {
    if ( x == '+' || x == '-' )
        return 1;
    elseif ( x == '*' || x == '/' )
        return 2;
    return 0;
}

```

```

char * Convert ( char * infix ) {
    struct stack *st;
    char *postfix = new char [strlen(infix)+1];
    int i=0, j=0;

    while ( infix[i] != '\0' ) {
        if ( isOperand ( infix[i] ) ) {
            postfix[j++] = infix[i++]; // char stored in j and both
        } else { // i, j incremented
            if ( precedence (infix[i]) > precedence (stackTop(st)) )
                push (&st, infix[i++]);
            else
                postfix[j++] = pop(&st);
        }
        while ( !isEmpty(st) ) {
            postfix[j++] = pop(&st);
        }
        postfix[j] = '\0';
        return postfix;
    }
}

```

Postfix Evaluation

$$\begin{aligned} \text{eg: } 3 * 5 + 6 / 2 - 4 & \xrightarrow{\text{BODMAS}} 15 + 3 - 4 \\ & = 18 - 4 \\ & = 14 \end{aligned}$$

$$\begin{aligned} \text{Postfix: } (35*) + (62/) - 4 \\ & = (35*62/+)-4 \\ & = 35*62/+4- \end{aligned}$$

1)

3

 push 3

2)

5
3

 push 5

3)

 pop two elements and perform operation. $3 * 5 = 15$

4)

15

 push 15

5)

6
15

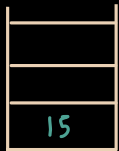
 push 6

6)

2
6
15

 push 2

7)

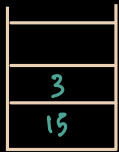


/ encountered.

$$6 / 2 = 3$$

pop 2 elements.

8)



push 3

9)

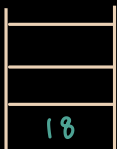


+ encountered.

$$15 + 3 = 18$$

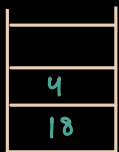
pop 3 and 15

10)



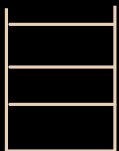
push 18

11)



push 4

12)

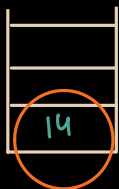


- encountered

$$18 - 4 = 14$$

pop 4 and 18

13)



push 14

→ result

Program

postfix	3	5	*	6	2	/	+	4	-	\0
	0	1	2	3	4	5	6	7	8	9

```
int Eval (char *postfix)
{
```

```
    struct stack st;
```

```
    int x1, x2, r;
```

```
    for (int i=0; postfix[i] != '\0'; i++)
```

```
    {
```

```
        if (isOperand(postfix[i]))
```

```
            push(&st, postfix[i] - '0');
```

```
        else
```

```
        {
```

```
            x2 = pop(&st);
```

```
            x1 = pop(&st);
```

```
            switch (postfix[i])
```

```
            {
```

```
                case '+': r = x1 + x2;
```

```
                push(&st, r);
```

```
                break;
```

```
                case '-': r = x1 - x2;
```

```
                push(&st, r);
```

```
                break;
```

// 3 when pushed
to stack, its ascii
code is pushed.

Type casting doesn't
help. Instead substr.
'0' from a no. in
ascii gives its int val.

eg: '3' - '0'

→ 51 - 48

= 3

